A.1 Mapping parallel work to a GPU

GPUs trade complex low-latency control for many execution units and high bandwidth. An individual GPU thread need not beat a CPU thread, and small workloads may not amortize launch/transfer costs. Programmable graphics and compute share processors, while rasterization, texturing, and ROP operations also use fixed functions. API pipeline stages need not own separate physical SM cores.

CUDA/Tesla terminology gives historical context. The 32-byte-sector, 32-bank, and occupancy examples explicitly use abstract parameters; combining them does not describe one GeForce 8800.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Interpolation in the graphics pipeline

Vertex shaders produce positions/attributes; clipping and rasterization produce fragments; fragment shading, depth/stencil, and blending update a framebuffer. A fragment need not become a visible pixel.

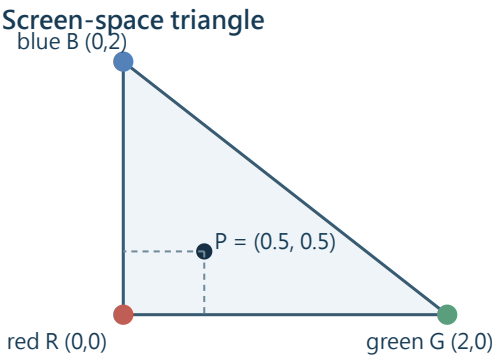
For screen vertices (0, 0),(2, 0),(0, 2), all clip w=1, the point (0.5, 0.5) has barycentric weights (1/2, 1/4, 1/4). They are nonnegative and sum to one. Pure R/G/B attributes interpolate to (0.5, 0.25, 0.25). For different w, correct perspective weights are

$$\frac{\lambda_i/w_i}{\sum_j \lambda_j/w_j}.$$

Screen-linear weights alone no longer suffice. Texture filtering reads neighboring texels; ROP may read old color/depth, so traffic exceeds one final color store.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Interpolate from vertex weights



P = ½R + ¼G + ¼B

Weights sum to 1 and are nonnegative: P is inside

Assume equal clip w = 1

Interpolated (R,G,B) = (0.5, 0.25, 0.25)

The point satisfies $P = \frac{1}{2}(0,0) + \frac{1}{4}(2,0) + \frac{1}{4}(0,2) = (0.5, 0.5)$. The nonnegative weights sum to 1, placing P inside the triangle. Applying them to pure red, green, and blue attributes gives $(0.5, 0.25, 0.25)$. This example assumes equal clip w. With the same screen weights and $w = (1, 2, 4)$, the unnormalized weights are $(1/2, 1/8, 1/16)$ with sum $11/16$; normalization gives $(8/11, 2/11, 1/11)$, a different interpolated result.

A.2 Host, device, and end-to-end costs

A discrete GPU has device DRAM and an I/O connection managed by drivers. Integrated/shared-memory designs can reduce copying but still require visibility, permissions, and bandwidth accounting. Unified addresses do not make movement free.

Assume sequential input/output copies without compute overlap. Input/output are 64 MiB each, link 8 GB/s each direction, launch 20 microseconds, kernel 2 ms:

$$T_{copies} = 2(64)(2^{20})/(8 \times 10^{9}) = 16.777216 \text{ ms}.$$

Total is 18.797216 ms. Against a 25 ms CPU, overall speedup is 1.330, not kernel-only 12.5. Retaining data for ten kernels costs 36.977216 ms with one input/output pair. Actual reuse depends on data dependencies and use cases.

A.3 Grid, block, and thread indices

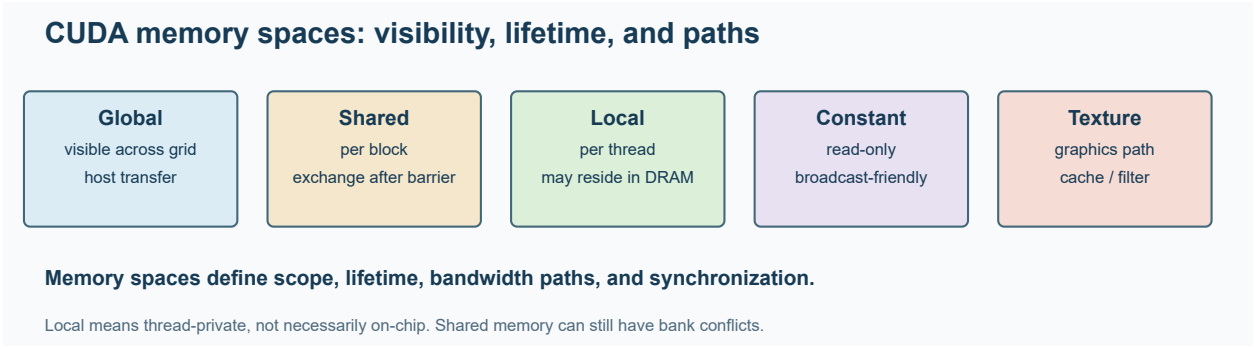
The mapping is $i = blockIdx.x\, blockDim.x + threadIdx.x$. N=1000, B=256 needs four blocks; the last has 232 valid outputs 768..999 and 24 inactive threads.

```
i = block_id * B + thread_id
if i < N:
    y[i] = f(x[i])
```

Every valid i has unique quotient/remainder (⌊i/B⌋, i mod B), proving coverage and exclusive writes. Skipping inactive threads is safe here because no barrier follows. With a collective barrier, predicate computation instead of returning required participants early.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Memory-space visibility and lifetime



Registers and local memory are thread-private; local arrays/spills usually reside in device DRAM. Shared memory is block-private scratch space lasting for the block. Global data can cross blocks but visibility follows synchronization rules. Constant memory can broadcast identical warp addresses. Texture paths have format/filter/cache semantics distinct from arbitrary loads. A memory-space name alone does not guarantee latency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

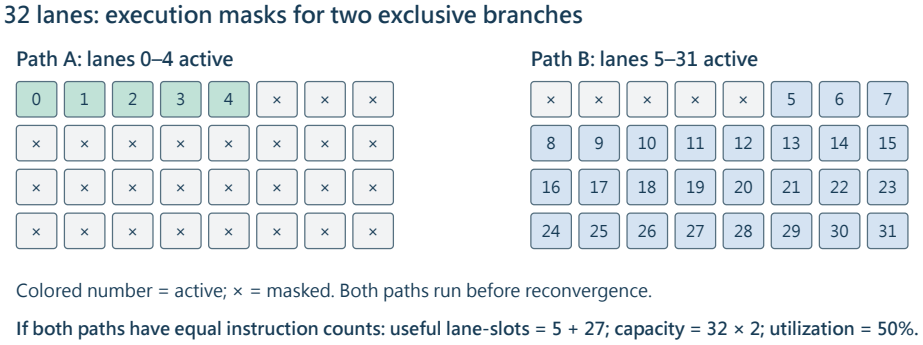
A barrier is more than a timer

A block barrier requires the specified participants and establishes memory visibility. It is not a general cross-block barrier. Apparent warp lockstep does not waive required synchronization.

For partial tiles, all threads predicate-load valid data or identities, reach a barrier, perform valid-output work, and reach another barrier. The second prevents a fast thread overwriting data still read by a slower one. Blocks need not run concurrently or in order. Resident blocks spinning for unscheduled blocks may occupy every resource and deadlock. Separate kernel phases provide a global completion boundary without that residency assumption.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.4 SMs, warps, and scoreboards



A warp issues together while each thread retains registers/logical state. Historical Tesla uses 32-thread warps and eight SPs, serving lanes across four core cycles. This throughput width does not make every result latency four cycles.

A scoreboard tracks dependencies; the scheduler issues ready warps. Other ready warps can cover a stalled warp, but high residency does not ensure readiness when all wait on a barrier or memory. PTX is a virtual ISA compiled/JITed for a target. One PTX instruction need not equal one cycle or one fixed-size SASS instruction.

A finite latency-hiding model

With one warp instruction issued every four cycles and a 32-cycle dependent chain per warp, eight ready warps give 32 cycles between same-warp issues. Four warps give 16 cycles and must wait 16 more. A 400-cycle dependency with only one outstanding chain per warp would need 100 warps, possibly beyond capacity. ILP, independent loads, caches, and memory parallelism change this requirement. Occupancy alone cannot guarantee zero stalls.

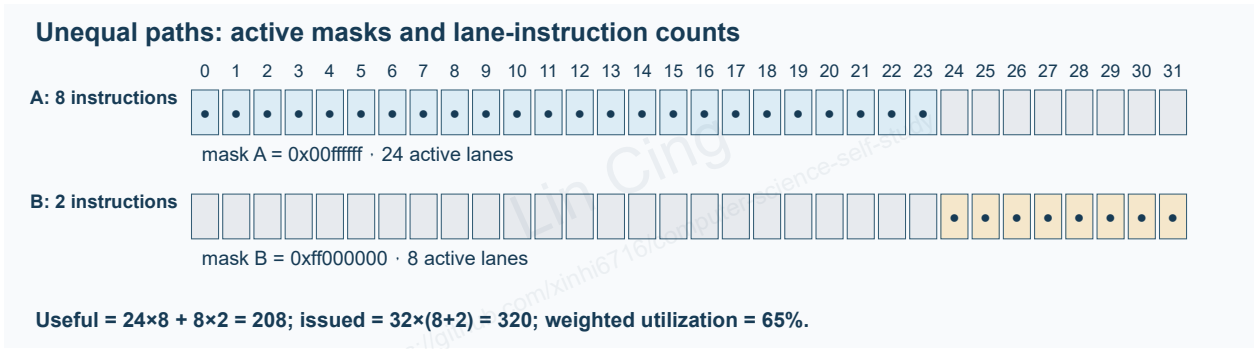
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Weighted divergence in lane slots

For W lanes, active counts a+b=W, and path lengths LA/LB, when both mutually exclusive paths execute,

$$\eta = \frac{aL_A + bL_B}{W(L_A + L_B)}.$$

Only equal path lengths reduce this to one-half. W=32, a=24, b=8, LA=8, LB=2 give 208 useful lane-instructions/320 slots=65%. This is not kernel speedup or DRAM efficiency.



A path with no active lane can be omitted from the denominator. Predication still consumes issue capacity; false-predicate memory lanes do not automatically generate DRAM transactions.

Occupancy is an integer resource problem

Use abstract limits 768 threads, 24 warps, eight blocks, 32768 registers, and 48 KiB shared per SM, warp size 32, ignoring allocation granularity. Blocks have 256 threads, 32 registers/thread, and 16 KiB shared:

$$blocks \le \min(\lfloor 768/256 \rfloor, 8, \lfloor 32768/(256(32)) \rfloor, \lfloor 48/16 \rfloor) = 3.$$

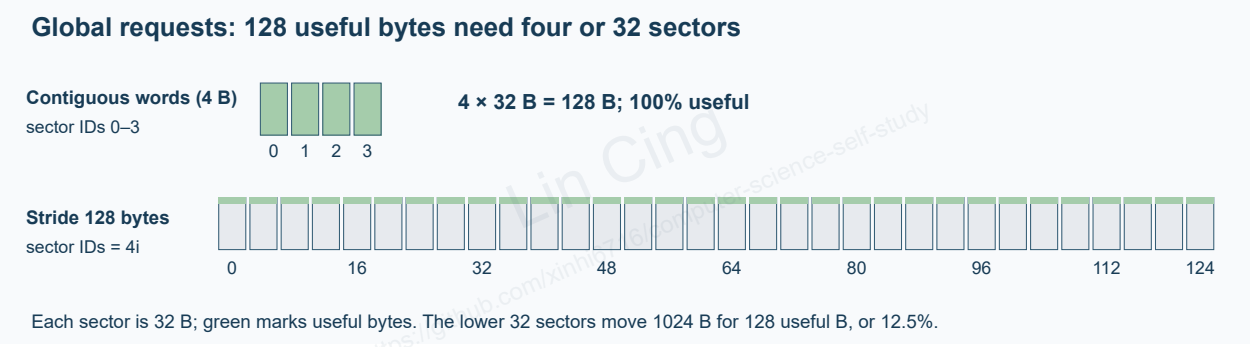
Three blocks give 24 warps, 100% occupancy. With 128 threads but 16 KiB shared/block, three blocks give 12 warps, 50%. Reducing shared to 8 KiB permits six blocks and 24 warps. For the original 256-thread version, 64 registers/thread instead limits to two blocks, 16/24 occupancy. These are residency bounds, not measured utilization. Reducing registers can cause spills and make higher occupancy slower.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.5 Counting coalesced sectors

Assume 32 lanes, four-byte words, and 32-byte sectors; count unique sectors for one warp request, ignoring cache/merging. This is not the complete GeForce 8800 rule set. For byte addresses ai, the sector set is $\{\lfloor a_i/32 \rfloor\}$.

Aligned contiguous 128 bytes use four sectors. Starting at +4 uses five, 160 transferred bytes for 128 useful, 80%. Stride 32 or 128 bytes gives 32 sectors, 1024 bytes, 12.5% useful. Equal sector counts need not imply equal channel/row locality.



All lanes reading one word request only four unique bytes, possibly served by one sector plus broadcast. Thirty-two logical uses do not become 128 distinct bytes.

Shared banks, broadcast, and padding

Assume 32 banks, four-byte words, bank=word index mod 32. Different banks serve in parallel; distinct words in one bank serialize. Identical-word reads can broadcast.

Column 0 of a 32 x 32 row-major array addresses 32r, so 32 distinct words collide in bank 0. Leading dimension 33 makes bank=r, using all banks.

Column access: each cell shows the bank chosen by one lane

stride 32: 32 distinct words map to bank 0

0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

stride 33: lane r maps to bank r

0	1	2	3	4	5	6	7
8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23
24	25	26	27	28	29	30	31

Assumes 32 banks and four-byte words; same-word broadcast is a different case.

Padding grows shared footprint 4096→4224 bytes and may reduce resident blocks at an integer threshold. Historical Tesla's 16-bank/half-warp rules require recounting 16 simultaneous requests; this 32-way result is not its measured behavior.

A.6 Floating-point order is part of the model

Binary32 has 24 significand bits. Choose representable $a = 1 + 2^{-23}$, $b = 1 - 2^{-23}$, and $c = -1$. The exact product is $ab = 1 - 2^{-46}$. Rounding that product to binary32 gives 1, so adding c then gives 0. A single-round IEEE FMA first computes the exact $ab + c = -2^{-46}$, which is exactly representable, so its result is -2^{-46}. Historical G80 MAD must not be assumed to have these FMA semantics. Subnormal, approximate-function, and exception support are version-specific.

Changing a reduction tree can change rounding. Binary32 `(10^8+1)+(-10^8)` gives 0 while `(10^8-10^8)+1` gives 1. An integer associativity proof does not establish identical floating-point bits. Specify error/reproducibility requirements separately.

A.7 Historical GeForce 8800 structure

The book's figure A.7.1 has eight TPCs, two SMs per TPC, and eight SPs per SM, total 128 SPs. Six 64-bit DRAM partitions provide a 384-bit interface. SM shared/constant/SFU paths differ from texture/ROP functions.

This structure check does not predict current GPU performance. Generations differ in caches, warps, FP, and transaction rules; equal core counts can have different clocks, mixes, and dataflow. ALU/SFU peaks can be counted together only when both architecture and workload permit that simultaneous use.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.8 CSR SpMV data

$$A = \begin{bmatrix} 2 & 0 & -1 & 0 \\ 0 & 3 & 0 & 4 \\ 5 & 0 & 0 & 6 \end{bmatrix}, \qquad x = (1, 2, 3, 4)^T.$$

Ap=[0, 2, 4, 6], Aj=[0, 2, 1, 3, 0, 3], Av=[2,−1, 3, 4, 5, 6]. Row i occupies `[Ap[i], Ap[i+1])`. Thus $y_0 = 2(1) - 1(3) = -1$, $y_1 = 3(2) + 4(4) = 22$, and $y_2 = 5(1) + 6(4) = 29$. Ap has rows+1 entries ending at nnz; Aj stays within column bounds. This rectangular matrix needs x length 4 and y length 3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One thread per row

```
row = block_id * B + thread_id
if row < rows:
    total = 0
    for k = Ap[row] .. Ap[row+1]-1:
        total += Av[k] * x[Aj[k]]
    y[row] = total
```

One writer owns each row. The invariant says total equals products already processed in that row's interval; termination gives the dot product. Empty rows output zero.

If 31 rows in a warp have one entry and one has 64, the simplified masked loop runs 64 rounds. There are 95 useful lane-iterations/2048 slots=4.64%. A warp-per-row mapping distributes a long row but can waste more lanes on short rows. Caching x helps only when reuse exceeds loading/synchronization costs.

Inclusive and exclusive scan

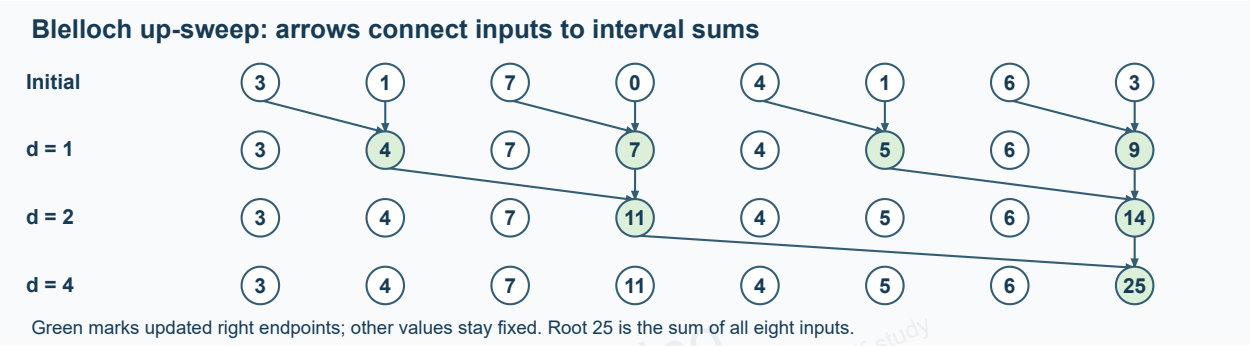
For [3, 1, 7, 0, 4, 1, 6, 3], inclusive scan is [3, 4, 11, 11, 15, 16, 22, 25]; exclusive scan is [0, 3, 4, 11, 11, 15, 16, 22]. Hillis–Steele reads each previous-round snapshot at distances 1, 2, 4:

Distance	Array after round
initial	[3, 1, 7, 0, 4, 1, 6, 3]
1	[3, 4, 8, 7, 4, 5, 7, 9]
2	[3, 4, 11, 11, 12, 12, 11, 14]
4	[3, 4, 11, 11, 15, 16, 22, 25]

Unsynchronized in-place writes may read current-round results. This case uses 7+6+4=17 adds; general work Θ(n log n), span Θ(log n). Small depth alone is not work efficiency.

Work-efficient up-sweep

Blelloch scan combines disjoint intervals in a binary tree.



d	Array
1	[3, 4, 7, 7, 4, 5, 6, 9]
2	[3, 4, 7, 11, 4, 5, 6, 14]
4	[3, 4, 7, 11, 4, 5, 6, 25]

Each internal node sums its two disjoint child intervals. Induction on height proves its interval sum. There are $n - 1$ additions and $\log_2 n$ levels.

Down-sweep carries preceding prefixes

Save total 25 and replace root with zero. If a parent stores the sum p before its interval and the left subtree sum is t, the left child gets p, the right gets p+t. These are precisely the prefixes before their respective intervals.

Step	Array
root=0	[3, 4, 7, 11, 4, 5, 6, 0]
d=4	[3, 4, 7, 0, 4, 5, 6, 11]
d=2	[3, 0, 7, 4, 4, 11, 6, 16]
d=1	[0, 3, 4, 11, 11, 15, 16, 22]

Induction down the tree gives each leaf the sum of earlier elements, proving exclusive scan. Down-sweep also takes n−1 adds, total 2(n−1) work and 2 log_2 n span, plus swaps/root setup. Pad non-power-of-two sizes with zero; N=0 returns empty output.

Shared-memory scan pseudocode

For power-of-two n, all participants execute every round; disjoint k positions run in parallel.

```
for d = 1, 2, 4, ..., n/2:
    parallel for k = 0, 2*d, 4*d, ...:
        x[k+2*d-1] += x[k+d-1]
    block_barrier()
if thread_id == 0:
    total = x[n-1]
    x[n-1] = 0
block_barrier()
for d = n/2, ..., 2, 1:
    parallel for k = 0, 2*d, 4*d, ...:
        t = x[k+d-1]
        p = x[k+2*d-1]
        x[k+d-1] = p
        x[k+2*d-1] = p + t
    block_barrier()
```

One designated thread saves total, then publishes it as needed. This in-place tree algorithm does not require Hillis–Steele's double-buffer snapshot; their synchronization rules differ.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Multiblock scan and global phase boundaries

For N=10, B=4, input [3, 1, 7, 0, 4, 1, 6, 3, 2, 5], zero-padded local scans give block sums [11, 14, 7]. Scanning these gives offsets [0, 11, 25]. Adding offsets yields [0, 3, 4, 11, 11, 15, 16, 22, 25, 27], total 32.

Run local scan/sums, scan sums, and add offsets as ordered phases. Recurse over many sums. Kernel completion supplies global order without simultaneous block residency. Fixed-size blocks produce a geometric series of per-level data sizes, so total work O (N), abstract tree span O (logN). Actual time includes launches, traffic, and finite-SM scheduling.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Reduction invariant and values

Merge the back half into the front with strides 4, 2, 1. Each active partial represents a disjoint subset; merging preserves that invariant and ends with the total.

Stride	Active partials
4	[3+4, 1+1, 7+6, 0+3]=[7, 2, 13, 3]
2	[7+13, 2+3]=[20, 5]
1	[20+5]=[25]

Seven adds complete the reduction. Multiple blocks can reduce block sums or use supported atomics. Atomic updates prevent lost additions but do not fix interblock floating-point summation order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Stable radix partition destinations

For unsigned three-bit keys [7, 2, 5, 0, 3, 4, 1, 6], bit 0 zero flags are [0, 1, 0, 1, 0, 1, 0, 1], exclusive prefixes [0, 0, 1, 1, 2, 2, 3, 3], total zeros Z=4. Zero destinations are Zi; one destinations are Z+i−Zi. Dest=[4, 0, 5, 1, 6, 2, 7, 3], producing [2, 0, 4, 6, 7, 5, 3, 1]. Prefixes increase in original order within each partition, so destinations are distinct, cover 0..7, and preserve stability.

Read each column as input index, zero flag, preceding zero count, and destination:

i	0	1	2	3	4	5	6	7
key	7	2	5	0	3	4	1	6
z_i	0	1	0	1	0	1	0	1
Z_i	0	0	1	1	2	2	3	3
dest	4	0	5	1	6	2	7	3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

LSD radix sort induction

Completed bits	Array
bit 0	[2, 0, 4, 6, 7, 5, 3, 1]
bits 0..1	[0, 4, 5, 1, 2, 6, 7, 3]
bits 0..2	[0, 1, 2, 3, 4, 5, 6, 7]

Assume the previous b low bits are sorted. Stable partition on bit b separates its values while preserving lower-bit order within each group, so b+1 bits are sorted. k passes of O (N) scan/scatter give O (kN) work. Separate input/output storage prevents overwrite. Signed/FP keys need a specified order-preserving transform; raw encoding need not match unsigned order.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

N-body acceleration and force

$$\mathbf{a}_i = G \sum_{j \neq i} \frac{m_j (\mathbf{x}_j - \mathbf{x}_i)}{(\|\mathbf{x}_j - \mathbf{x}_i\|^2 + \varepsilon^2)^{3/2}}.$$

Target mass belongs in force $m_i \mathbf{a}_i$, not acceleration's numerator. Softening changes the physical model as well as avoiding singularity. For x=[0, 1, 3, 4], unit masses/G, epsilon=0, and omitted self terms:

$$a_0 = 1 + 1/9 + 1/16 = 169/144, \quad a_1 = -1 + 1/4 + 1/9 = -23/36, \quad a_2 = 23/36, \quad a_3 = -169/144.$$

Total force zero checks pairwise signs; finite precision/order can introduce small error.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

N-body tiling and two barriers

Each B-thread block owns B targets and cooperatively loads B sources into shared memory.

```
i = block_id * B + thread_id
my = (i < N) ? body[i] : zero_body
acc = 0
for base = 0; base < N; base += B:
    j = base + thread_id
    sh[thread_id] = (j < N) ? body[j] : zero_body
    block_barrier()
    if i < N:
        for k = 0 .. min(B,N-base)-1:
            if base+k != i:
                acc += acceleration(my, sh[k])
    block_barrier()
if i < N:
    output[i] = acc
```

The first barrier completes tile publication; the second prevents early reuse. Inactive targets still help load and participate. Each valid target processes every valid source other than itself once, matching the direct mathematical sum. Pair work remains $\Theta(N^2)$.

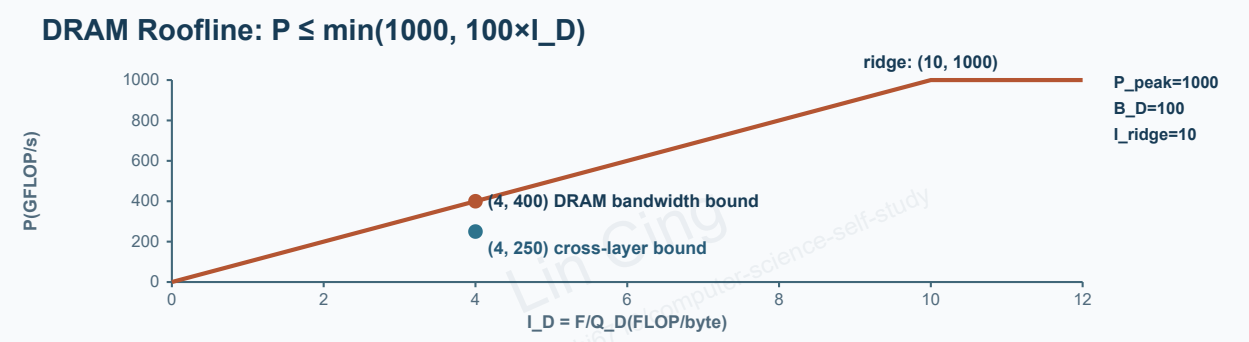
Which traffic tiling reduces

Assume N divisible by B, no cache reuse, one separate my load per target, and N source loads before skipping self computation. Naive sources=N^2; tiled=N/B blocks each loading N sources, total N^2/B.

At N=4, B=2, sources fall 16→8, but four my loads make totals 20→12, only a 40% total reduction rather than an exact halving. Compiler/cache reuse and body-field widths require interface-specific byte recounting. Shared sources may still be read by B targets, and output traffic remains. Reduced DRAM traffic does not reduce every hierarchy's bytes equally.

Do not mix traffic boundaries

Fix one phase with $F = 10^9$ FLOPs and sustained compute ceiling $P_C = 1000$ GFLOP/s. It transfers $Q_D = 0.25 \times 10^9$ bytes through DRAM at $B_D = 100$ GB/s and $Q_S = 4 \times 10^9$ bytes through shared memory at $B_S = 1000$ GB/s. These are declared model inputs, not measured specifications of one GPU.

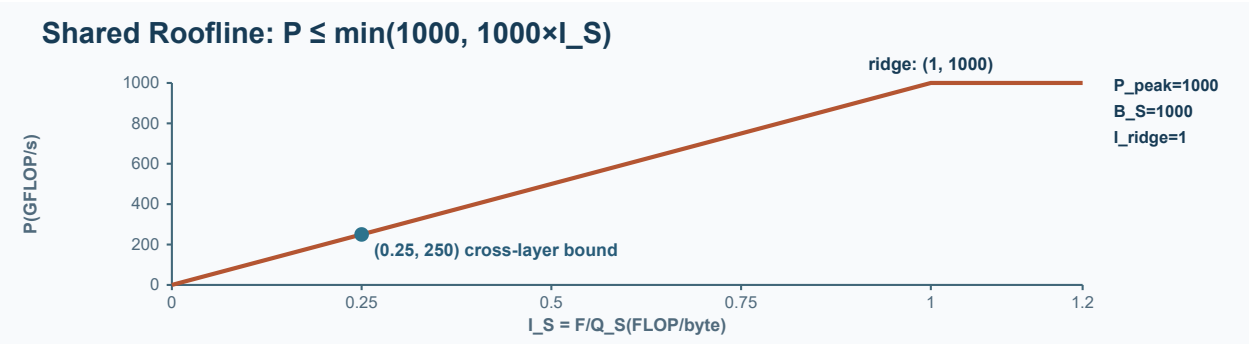


$$I_D = F/Q_D = 10^9/(0.25 \times 10^9) = 4 \text{ FLOP/byte},$$

$$P_D \le \min(P_C, I_D B_D) = \min(1000, 4 \cdot 100) = 400 \text{ GFLOP/s}.$$

The DRAM ridge is $P_C/B_D = 10$ FLOP/byte. Its independent working point $(4, 400)$ lies on the bandwidth slope.

Do not mix traffic boundaries (continued)

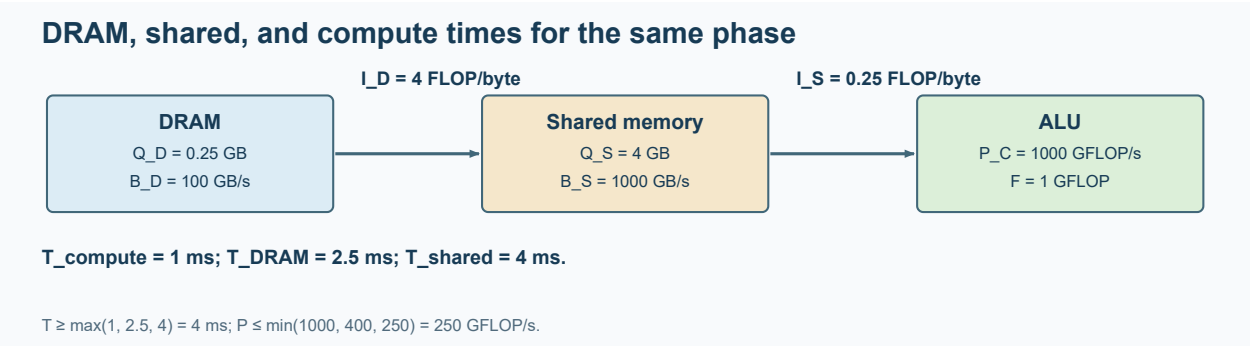


$$I_S = F/Q_S = 10^9/(4 \times 10^9) = 0.25 \text{ FLOP/byte},$$

$$P_S \le \min(P_C, I_S B_S) = \min(1000, 0.25 \cdot 1000) = 250 \text{ GFLOP/s}.$$

The shared ridge is 1 FLOP/byte. Its plateau remains 1000 GFLOP/s; $(0.25, 250)$ lies on the slope. Drawing 250 as the entire horizontal shared roof would change the model.

Do not mix traffic boundaries (continued)



The three resource times are

$$T_C = 10^9/(1000 \times 10^9) = 0.001\ \text{s},$$

$$T_D = 0.25 \times 10^9/(100 \times 10^9) = 0.0025\ \text{s}, \qquad T_S = 4 \times 10^9/(1000 \times 10^9) = 0.004\ \text{s}.$$

Even if their traffic overlaps completely,

$$T \geq \max(1, 2.5, 4) = 4\ \text{ms}, \qquad P \leq F/T = 10^9/0.004 = 250\ \text{GFLOP/s}.$$

At $(I_D, I_S) = (4, 0.25)$, the combined model bound is 250 GFLOP/s. Bank replay, synchronization, occupancy limits, and extra traffic may lower actual throughput. Use each interface's own byte denominator and sustainable bandwidth for this same phase.

Do not mix traffic boundaries (continued)

Resource	Bytes	Intensity	Sustained ceiling or bandwidth	Resource bound
Compute	Not applicable	Not applicable	1000 GFLOP/s	1000 GFLOP/s
DRAM	0.25×10^9	4 FLOP/byte	100 GB/s	400 GFLOP/s
Shared	4×10^9	0.25 FLOP/byte	1000 GB/s	250 GFLOP/s
Combined	Separate per interface	Separate per interface	Separate per interface	250 GFLOP/s

Lin Cing https://github.com/xinhi6716/computer-science-self-study

A.9 Counterexamples to performance claims

Reducing register count can cause spills and additional traffic despite higher occupancy. Different precisions, approximate functions, FMA counting, or transfer endpoints also make equal reported FLOP counts incomparable. High occupancy can coexist with stalls; fewer sectors can coexist with bank/channel imbalance. A roof is an upper bound, not an attained rate. If AI 0.3 and 320 GB/s imply 96 GFLOP/s while 1.2 GFLOP in 5 ms suggests 240, the phase, units, FLOP count, and interface bytes cannot all describe the same experiment. Added divergence cannot explain exceeding an upper bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.10 Correctness before mapping

Prove unique output ownership, valid indices, collective participation, and phase ordering before comparing mappings. For scan/radix, the interval-prefix and stability proofs justify all legal integer inputs; for N-body, each target/source pair and mass factor must be correct. Then compare measured end-to-end time including transfer, launch, synchronization, traffic, and numerical requirements.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

A.11 Historical architecture and portable methods

GPUs evolved from fixed graphics stages toward unified programmable processors. CUDA and PTX map many logical threads onto scalable resources. Warp width, banks, caches, and numerical support can change while latency, throughput, capacity, and visibility remain distinct contracts. Historical Tesla/GeForce examples explain design constraints rather than current rankings. Mapping, resource floors, explicit memory boundaries, and complete numerical traces transfer across generations only after updating the architecture parameters.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References

Patterson and Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Revised Fourth Edition, Appendix A. Historical CUDA/Tesla cases and the separately declared sector/bank/occupancy models retain their distinct assumptions. See the unit guide, exercises, and solutions.

The source map covers A.1–A.11, printed A-3–A-77, PDF 732–806. Warp scheduling/divergence appear at A-28–A-30 (PDF 757–759), the 16-bank case at A-40 (PDF 769), GeForce figure A.7.1 at A-47 (PDF 776), and scan/reduction/radix/N-body at A-60–A-69 (PDF 789–798). See Roofline DOI and NERSC interface-byte definitions.

References and attribution

David A. Patterson and John L. Hennessy, Computer Organization and Design, Revised Fourth Edition, Appendix A; computer organization and structure (CSO) course slides.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study